

Глава 9

Итераторы STL

В главе подробно описываются итераторы, в частности, категории итераторов, операции над ними, адаптеры итераторов и пользовательские итераторы.

9.1. Заголовочные файлы для итераторов

Все контейнеры определяют свои собственные типы итераторов, поэтому для использования итераторов в контейнерах нет необходимости в специальном заголовочном файле. Однако некоторые определения специальных итераторов, например обратных итераторов, и некоторые функции для итераторов содержатся в заголовочном файле `<iterator>`. Некоторые реализации включают этот заголовочный файл при использовании любых контейнеров, чтобы определить для них типы обратных итераторов. Однако это препятствует переносимости программ. Следовательно, для того чтобы использовать необычные контейнерные итераторы и их типы, необходимо вставить в программу этот файл.

9.2. Категории итераторов

Итераторы — это объекты, которые могут обходить элементы последовательности с помощью универсального интерфейса, напоминающего интерфейс обычных указателей (см. раздел 6.3). Итераторы следуют принципу чистой абстракции: все, что *ведет* себя как итератор, *является* итератором. Однако итераторы наделены разными возможностями, которые имеют большое значение, потому что некоторые алгоритмы предъявляют к итераторам особые требования. Например, алгоритмы сортировки требуют, чтобы итераторы обеспечивали прямой доступ, потому что в противном случае быстрдействие программы может снизиться. По этой причине итераторы разделяются на категории (рис. 9.1). Возможности этих категорий перечислены в табл. 9.1 и обсуждаются в следующих подразделах.

Считывающие итераторы, которые могут также записывать данные, называются *модифицирующими* (например, *модифицирующий последовательный итератор*).

9.2.1. Итераторы вывода

Итераторы вывода могут перемещаться только вперед и допускают запись. Следовательно, с их помощью можно лишь последовательно присваивать элементам новые значения. Итератор вывода нельзя использовать для повторного обхода одного и того же диапазона. Фактически даже не гарантируется, что возможно повторное присваивание без увеличения итератора. Эти итераторы предназначены для записи значения в “черную дыру” следующим образом:

```
while (...) {
    *pos = ...; // присваиваем значение
    ++pos;     // перемещаемся вперед (готовимся к следующему присваиванию)
}
```



Рис. 9.1. Категории итераторов

Таблица 9.1. Обзор возможностей итераторов

Категория итераторов	Возможность	Поддержка
Итератор вывода	Запись в прямом направлении	Поток вывода, итератор вставки
Итератор ввода	Чтение в прямом направлении	Поток ввода
Однонаправленный итератор	Чтение в прямом направлении	Последовательный список, неупорядоченные контейнеры
Двунаправленный итератор	Чтение в прямом и обратном направлениях	Список, множество, мультимножество, отображение, мультиотображение
Итератор произвольного доступа	Чтение с произвольным доступом	Массив, вектор, дек, строка, массив в стиле языка C

В табл. 9.2 перечислены допустимые операции над итераторами вывода. Единственное допустимое использование операции `*` — в левой части оператора присваивания.

Таблица 9.2. Операции над итераторами вывода

Выражение	Результат
<code>*iter = val</code>	Записывает значение <code>val</code> в то место, на которое ссылается итератор
<code>++iter</code>	Перемещает итератор вперед (возвращает новую позицию)
<code>iter++</code>	Перемещает итератор вперед (возвращает старую позицию)
<code>TYPE (iter)</code>	Копирует итератор (копирующий конструктор)

К итераторам вывода не применяются операции сравнения. Невозможно проверить, является ли итератор вывода корректным и была ли запись успешной. Можно только записывать, записывать и записывать значения. Как правило, конец записи определяется дополнительным требованием к конкретным итераторам вывода.

Часто итераторы могут и читать, и записывать значения. По этой причине все итераторы, выполняющие чтение, могут иметь дополнительную возможность записи. В этом случае они называются *модифицирующими*.

Типичным примером “чистого” итератора вывода является итератор, выполняющий запись в стандартный поток вывода (например, на экран или на принтер). Если вы используете два итератора вывода для записи на экран, то второе слово следует за первым, а не заменяет его. Другим типичным примером итераторов вывода являются итераторы вставки. Итераторы вставки — это итераторы, вставляющие значения в контейнеры. Присвоить им значение — значит вставить его. При записи второго значения первое не заменяется; второе значение просто вставляется, как и первое. Итераторы вставки рассматриваются в разделе 9.4.2.

Все константные итераторы, предусмотренные в контейнерах, а также их функции-члены `cbegin()` и `cend()` не являются итераторами вывода, потому что они не позволяют выполнять запись в позиции, на которые указывают эти итераторы.

9.2.2. Итераторы ввода

Итераторы ввода могут только последовательно обходить элемент за элементом, выполняя чтение. Следовательно, они возвращают значения поэлементно. Операции над итераторами ввода приведены в табл. 9.3.

Таблица 9.3. Операции над итераторами ввода

Выражение	Результат
<code>*iter</code>	Обеспечивает доступ для чтения к текущему элементу
<code>iter -> member</code>	Обеспечивает доступ для чтения к члену текущего элемента
<code>++iter</code>	Перемещает итератор вперед (возвращает новую позицию)
<code>iter++</code>	Перемещает итератор вперед
<code>iter1 == iter2</code>	Возвращает результат проверки того, что два итератора являются равными
<code>iter1 != iter2</code>	Возвращает результат проверки того, что два итератора не являются равными
<code>TYPE(iter)</code>	Копирует итератор (копирующий конструктор)

Итераторы ввода могут читать элементы только один раз. Следовательно, если вы копируете итератор ввода и выполняете чтение с помощью оригинального итератора и его копии, то можете получить разные значения.

Все итераторы, ссылающиеся на значения, которые подлежат обработке, должны быть итераторами ввода. Однако обычно они могут больше. Типичным примером “чистого” итератора ввода является итератор, выполняющий чтение из стандартного потока ввода, которым, как правило, служит клавиатура. Одно и то же значение нельзя прочитать дважды. Как только слово считано из потока ввода, итератор переходит к следующему слову.

Для итераторов ввода операции `==` и `!=` выполняют лишь проверку, равен ли итератор *запредельному итератору* (past-the-end iterator). Это необходимо потому, что операции над итераторами ввода обычно выполняются следующим образом:

```
InputIterator pos, end;

while (pos != end) {
```

```
... // доступ только для чтения с помощью *pos
++pos;
}
```

Нет гарантии, что два разных итератора, которые оба не равны запредельному итератору, при сравнении окажутся разными, если они ссылаются на разные позиции. (Это требование вводится начиная с однонаправленных итераторов.)

Отметим также, что итераторы ввода не требуют, чтобы оператор *iter++* что-то возвращал; однако обычно он возвращает старую позицию итератора.

Всегда следует отдавать предпочтение префиксному оператору инкремента, а не постфиксному, потому что в общем случае он работает быстрее. Это объясняется тем, что префиксному инкременту не приходится возвращать старое значение, которое должно храниться во временном объекте. Следовательно, для любого итератора *pos* (и любого абстрактного типа) следует выбрать

```
++pos // хорошо и быстро
```

а не

```
pos++ // хорошо, но не так быстро
```

То же самое касается операторов декремента, если таковые определены (для итераторов ввода они не предусмотрены).

9.2.3. Однонаправленные итераторы

Однонаправленные итераторы — это итераторы ввода, обеспечивающие дополнительные гарантии при чтении в прямом направлении. Операции над однонаправленными итераторами приведены в табл. 9.4.

Таблица 9.4. Операции над однонаправленными итераторами

Выражение	Результат
<i>*iter</i>	Обеспечивает доступ к текущему элементу
<i>iter -> member</i>	Обеспечивает доступ к члену текущего элемента
<i>++iter</i>	Перемещает итератор вперед (возвращает новую позицию)
<i>iter++</i>	Перемещает итератор вперед (возвращает старую позицию)
<i>iter1 == iter2</i>	Возвращает результат проверки того, что два итератора являются равными
<i>iter1 != iter2</i>	Возвращает результат проверки того, что два итератора не являются равными
<i>TYPE()</i>	Создает итератор (конструктор по умолчанию)
<i>TYPE(iter)</i>	Копирует итератор (копирующий конструктор)
<i>iter1 = iter2</i>	Присваивает итератор

В отличие от итератора ввода, гарантируется, что для двух однонаправленных итераторов, ссылающихся на один и тот же элемент, операция *==* возвращает значение *true* и что после инкремента обоих итераторов они будут ссылаться на один и тот же элемент. Рассмотрим пример:


```
ForwardIterator pos1, pos2;
```

```
pos1 = pos2 = begin; // оба итератора ссылаются на один и тот же элемент
if (pos1 != end) {
    ++pos1;           // итератор pos1 смещается на один элемент вперед
    while (pos1 != end) {
        if (*pos1 == *pos2) {
            ...       // обработка соседних дубликатов
            ++pos1;
            ++pos2;
        }
    }
}
```

Однонаправленные итераторы предусмотрены в следующих объектах и типах:

- класс `forward_list<>`;
- неупорядоченные контейнеры.

В то же время в неупорядоченных контейнерах библиотеки разрешают использовать вместо однонаправленных двунаправленные итераторы (см. раздел 7.9.1).

Однонаправленный итератор, удовлетворяющий требованиям итератора вывода, является *модифицирующим* однонаправленным итератором и может использоваться как для чтения, так и для записи.

9.2.4. Двунаправленные итераторы

Двунаправленные итераторы — это однонаправленные итераторы, обладающие дополнительной возможностью перемещаться по элементам в обратном направлении. Следовательно, они предусматривают оператор декремента для перемещения назад (табл. 9.5).

Таблица 9.5. Дополнительные операции над двунаправленными итераторами

Выражение	Результат
<code>--iter</code>	Перемещает итератор назад (возвращает новую позицию)
<code>iter--</code>	Перемещает итератор назад (возвращает старую позицию)

Двунаправленные итераторы предусмотрены в следующих объектах и классах:

- класс `list<>`;
- ассоциативные контейнеры.

Двунаправленный итератор, удовлетворяющий требованиям итератора вывода, является *модифицирующим* двунаправленным итератором и может использоваться как для чтения, так и для записи.

9.2.5. Итераторы произвольного доступа

Итераторы произвольного доступа обладают всеми возможностями двунаправленных итераторов и произвольным доступом к элементам. Следовательно, они реализуют операции *арифметики итераторов* (по аналогии с *арифметикой обычных указателей*). Иначе

говоря, к ним можно прибавлять и вычитать смещения, вычислять их разности и сравнивать итераторы с помощью операторов сравнения, например `<` и `>`. В табл. 9.6 приведены дополнительные операции над итераторами прямого доступа.

Таблица 9.6. Операции над итераторами прямого доступа

Выражение	Результат
<code>iter[n]</code>	Обеспечивает доступ к элементу с индексом <i>n</i>
<code>iter+=n</code>	Перемещает итератор на <i>n</i> элементов вперед (или назад, если <i>n</i> отрицательное)
<code>iter-=n</code>	Перемещает итератор на <i>n</i> элементов назад (или вперед, если <i>n</i> отрицательное)
<code>iter+n</code>	Возвращает итератор, установленный на элемент, расположенный на <i>n</i> позиций вперед
<code>n+iter</code>	Возвращает итератор, установленный на элемент, расположенный на <i>n</i> позиций вперед
<code>iter-n</code>	Возвращает итератор, установленный на элемент, расположенный на <i>n</i> позиций назад
<code>iter1-iter2</code>	Возвращает расстояние между итераторами <i>iter1</i> и <i>iter2</i>
<code>iter1<iter2</code>	Возвращает результат проверки того, что итератор <i>iter1</i> предшествует итератору <i>iter2</i>
<code>iter1>iter2</code>	Возвращает результат проверки того, что итератор <i>iter2</i> предшествует итератору <i>iter1</i>
<code>iter1<=iter2</code>	Возвращает результат проверки того, что итератор <i>iter2</i> не предшествует итератору <i>iter1</i>
<code>iter1>=iter2</code>	Возвращает результат проверки того, что итератор <i>iter1</i> предшествует итератору <i>iter2</i>

Итераторы произвольного доступа предусмотрены в следующих объектах и типах:

- контейнеры с произвольным доступом (`array`, `vector`, `deque`);
- строки (`string`, `wstring`);
- обычные массивы в стиле C (указатели).

Следующая программа иллюстрирует специальные возможности итераторов произвольного доступа:

```
// iter/itercategory1.cpp

#include <vector>
#include <iostream>
using namespace std;

int main()
{
    vector<int> coll;

    // вставляем элементы от -3 до 9
    for (int i=-3; i<=9; ++i) {
        coll.push_back (i);
```

```

}

// выводим на экран количество элементов,
// вычисляя расстояние между концом и началом вектора
// - ПРИМЕЧАНИЕ: к итераторам применяется оператор -
cout << "number/distance: " << coll.end()-coll.begin() << endl;

// выводим на экран все элементы
// - ПРИМЕЧАНИЕ: к итераторам применяется оператор <, а не !=
vector<int>::iterator pos;
for (pos=coll.begin(); pos<coll.end(); ++pos) {
    cout << *pos << ' ';
}
cout << endl;

// выводим на экран все элементы
// - ПРИМЕЧАНИЕ: к итераторам применяется оператор [], а не *
for (int i=0; i<coll.size(); ++i) {
    cout << coll.begin()[i] << ' ';
}
cout << endl;

// выводим на экран каждый второй элемент
// - ПРИМЕЧАНИЕ: используется оператор +=
for (pos = coll.begin(); pos < coll.end()-1; pos += 2) {
    cout << *pos << ' ';
}
cout << endl;
}

```

Программа выводит следующий результат.

```

number/distance: 13
-3 -2 -1 0 1 2 3 4 5 6 7 8 9
-3 -2 -1 0 1 2 3 4 5 6 7 8 9
-3 -1 1 3 5 7

```

Эта программа не работает с (последовательными) списками, (неупорядоченными) множествами и (неупорядоченными) отображениями, потому что все операции, упомянутые в примечаниях, предусмотрены только для итераторов произвольного доступа. В частности, следует помнить, что оператор `<` можно использовать в качестве критерия остановки цикла только при работе с итераторами произвольного доступа.

Отметим, что следующее выражение в последнем цикле требует, чтобы вектор `coll` содержал хотя бы один элемент:

```
pos < coll.end()-1
```

Если коллекция пустая, выражение `coll.end()-1` может вернуть позицию, предшествующую `coll.begin()`. Сравнение может по-прежнему выполняться, но, строго говоря, перемещение итератора на позицию, предшествующую началу контейнера, приводит к непредсказуемым последствиям. Аналогично выражение `pos += 2` может привести к непредсказуемым последствиям, если оно перемещает итератор за позицию `end()`. Таким образом, изменение последнего цикла на следующий фрагмент может оказаться очень

опасным, поскольку приводит к непредсказуемым последствиям, если коллекция содержит нечетное количество элементов (рис. 9.2).

```
for (pos = coll.begin(); pos < coll.end(); pos += 2) {
    cout << *pos << ' ';
}
```

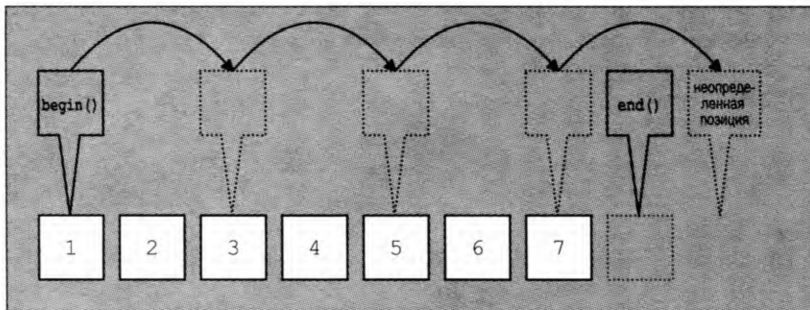


Рис. 9.2. Перемещение итераторов вправо на несколько позиций

Итератор произвольного доступа, удовлетворяющий требованиям итератора вывода, является *модифицирующим* итератором произвольного доступа и может использоваться как для чтения, так и для записи.

9.2.6. Проблема инкремента и декремента итераторов вектора

Использование операторов инкремента и декремента итераторов создает странную проблему. В общем случае можно применять инкремент и декремент к временным итераторам. Однако для объектов класса `array`, векторов и строк на некоторых платформах такие программы могут не компилироваться. Рассмотрим следующий пример:

```
std::vector<int> coll;

...
// сортируем, начиная со второго элемента
// - версия, не допускающая перенос на другие компьютеры
if (coll.size() > 1) {
    std::sort(++coll.begin(), coll.end());
}
```

В зависимости от реализации компиляция выражения `++coll.begin()` может закончиться неудачей. Однако если вы используете, скажем, дек, а не вектор, то компиляция пройдет успешно.

Причина этой странной проблемы заключается в том, что итераторы векторов, массивов и строк могут быть реализованы, как обычные указатели. Как и при работе с любыми другими элементарными типами, такими как указатели, модифицировать временные значения нельзя. В то же время для структур и классов это разрешено. Таким образом, если итератор реализован как обычный указатель, компиляция заканчивается неудачей; если же он реализован в виде класса, компиляция проходит успешно.

Вследствие этого предыдущий код всегда работает со всеми контейнерами (за исключением класса `array`, векторов и строк), потому что для контейнеров нельзя реализовать итераторы как обычные указатели. Однако для массивов, векторов и строк работоспособность кода зависит от реализации. Часто используются обычные указатели. Если же вы, к примеру, используете “безопасную версию” библиотеки STL (что случается все чаще), то итераторы реализуются как классы.

Для обеспечения переносимости стандарт C++11 предусмотрел вспомогательную функцию `next()` (см. раздел 9.3.2), поэтому можно написать следующий код:

```
std::vector<int> coll;

...
// сортируем начиная со второго элемента
// - переносимо начиная со стандарта C++11
if (coll.size() > 1) {
    std::sort(std::next(coll.begin()), coll.end());
}
```

До появления стандарта C++11 следовало использовать вспомогательный объект:

```
std::vector<int> coll;

...
// сортируем начиная со второго элемента
// - переносимо до появления стандарта C++11
if (coll.size() > 1) {
    std::vector<int>::iterator beg = coll.begin();
    std::sort(++beg, coll.end());
}
```

Эта проблема не такая страшная, как кажется. Непредвиденных последствий не наступает, потому что проблема выявляется на этапе компиляции. И все же досадно тратить время на ее решение.

9.3. Вспомогательные функции для работы с итераторами

Стандартная библиотека содержит несколько вспомогательных функций для работы с итераторами: `advance()`, `next()`, `prev()`, `distance()` и `iter_swap()`. Первые четыре функции наделяют все итераторы некоторыми возможностями, которыми обладают только итераторы произвольного доступа: перемещаться на один элемент вперед (или назад) и вычислять разность между итераторами. Последняя вспомогательная функция позволяет обменивать значения двух итераторов.

9.3.1. Функция `advance()`

Функция `advance()` переносит итератор вперед на указанное аргументом количество позиций. Следовательно, эта функция позволяет итератору перемещаться вперед (или назад) на несколько элементов.

```
#include <iterator>
void advance (InputIterator& pos, Dist n)
```

- Переносит итератор ввода *pos* на *n* элементов вперед (или назад).
- Для двунаправленных итераторов и итераторов произвольного доступа число *n* может быть отрицательным, задавая перемещение назад.
- *Dist* — шаблонный тип. Обычно он должен быть целочисленным типом, потому что к нему применяются такие операции, как *<*, *++*, *--*, и сравнения с 0.
- Отметим, что функция *advance()* не проверяет, заходит ли она за позицию *end()* последовательности (она не может это проверить, потому что итераторы вообще не знают ничего о контейнерах, которые они обходят). Следовательно, вызов этих функций может привести к непредсказуемым последствиям, потому что результат применения операции *++* к концу последовательности не определен.

Благодаря свойствам итераторов (см. раздел 9.5) эта функция всегда использует наилучшую реализацию, зависящую от категории итератора. Для итераторов произвольного доступа она просто выполняет вызов *pos+=n*. Следовательно, для таких итераторов функция *advance()* имеет константную сложность. Для всех других итераторов она *n* раз выполняет оператор *++pos* (или *--pos*, если *n* отрицательное). Таким образом, для всех других категорий итераторов функция *advance()* имеет линейную сложность.

Для того чтобы иметь возможность изменять контейнеры и типы итераторов, следует использовать функцию *advance()*, а не операцию *+=*. Однако, делая это, следует учитывать риск непреднамеренного снижения производительности программы. Дело в том, что вы не обнаружите никакого снижения производительности, пока не воспользуетесь контейнерами, не предоставляющими итераторы произвольного доступа (низкое быстродействие — одна из причин, по которым оператор *+=* предусмотрен только для итераторов произвольного доступа). Отметим также, что функция *advance()* ничего не возвращает, а оператор *+=* возвращает новую позицию и поэтому может быть частью более крупного выражения. Рассмотрим пример использования функции *advance()*:

```
// iter/advance1.cpp

#include <iterator>
#include <iostream>
#include <list>
#include <algorithm>
using namespace std;

int main()
{
    list<int> coll;

    // вставляем элементы от 1 до 9
    for (int i=1; i<=9; ++i) {
        coll.push_back(i);
    }

    list<int>::iterator pos = coll.begin();

    // выводим на экран текущий элемент
```

```

cout << *pos << endl;

// перемещаемся на три элемента вперед
advance (pos, 3);

// выводим на экран текущий элемент
cout << *pos << endl;

// перемещаемся на один элемент назад
advance (pos, -1);

// выводим на экран текущий элемент
cout << *pos << endl;
}

```

В этой программе функция `advance()` позволяет переместить итератор `pos` на три элемента вперед и на один элемент назад. Следовательно, результат работы программы будет таким:

```

1
4
3

```

Другой способ использования функции `advance()` — игнорирование некоторых данных, получаемых итераторами при чтении потока ввода. См. пример в разделе 9.4.3.

9.3.2. Функции `next()` и `prev()`

В соответствии со стандартом C++11 две эти вспомогательные функции позволяют перемещать итератор в следующую или предыдущую позицию:

```

#include <iterator>
ForwardIterator next (ForwardIterator pos)
ForwardIterator next (ForwardIterator pos, Dist n)

```

- Возвращают позицию прямого итератора `pos`, в которую он попал бы, если бы переместился на одну или `n` позиций вперед.
- Для двунаправленных итераторов и итераторов произвольного доступа число `n` может быть отрицательным и приводить к перемещению в предыдущие позиции.
- `Dist` — это тип `std::iterator_traits<ForwardIterator>::difference_type`.
- При выполнении функции выполняется вызов `advance(pos, n)` для внутреннего временного объекта.
- Функция `next()` не проверяет факт выхода за позицию `end()` последовательности. Следовательно, именно вызывающая сторона должна гарантировать корректность результата.

```

#include <iterator>
BidirectionalIterator prev (BidirectionalIterator pos)
BidirectionalIterator prev (BidirectionalIterator pos, Dist n)

```

- Возвращают позицию двунаправленного итератора *pos*, в которую он попал бы, если бы переместился на одну или *n* позиций назад.
- Для перехода на последующие позиции число *n* может быть отрицательным.
- `Dist` — это тип `std::iterator_traits<BidirectionalIterator>::difference_type`.
- При выполнении функции выполняется вызов `advance(pos, -n)` для внутреннего временного объекта.
- Отметим, что функция `prev()` не проверяет факт выхода за позицию `begin()` последовательности. Следовательно, именно вызывающая сторона должна гарантировать корректность результата.

Эти функции позволяют, например, обойти коллекцию, проверяя значения следующих элементов:

```
auto pos = coll.begin();
while (pos != coll.end() && std::next(pos) != coll.end()) {
    ...
    ++pos;
}
```

Это особенно полезно, учитывая, что прямые и двунаправленные итераторы не предусматривают операции `+` и `-`. В противном случае нам всегда требовался бы временный объект

```
auto pos = coll.begin();
auto nextPos = pos;
++nextPos;
while (pos != coll.end() && nextPos != coll.end()) {
    ...
    ++pos;
    ++nextPos;
}
```

или код, использующий исключительно итераторы произвольного доступа.

```
auto pos = coll.begin();
while (pos != coll.end() && pos+1 != coll.end()) {
    ...
    ++pos;
}
```

Не забудьте проверить, что позиция является корректной, перед тем как использовать ее (по этой причине мы сначала проверяем, не равен ли итератор `pos` итератору `coll.end()`, прежде чем перейти на следующую позицию).

Другое приложение функций `next()` и `prev()` — возможность не писать выражения, такие как `++coll.begin()`, для работы со вторым элементом коллекции. Проблема заключается в том, что использование выражения `++coll.begin()` вместо `std::next(coll.begin())` может вызвать ошибку компиляции (см. раздел 9.2.6).

Третье применение функции `next()` — это работа с классом `forward_list` и функцией `before_begin()` (см. раздел 7.6.2).

9.3.3. Функция `distance()`

Функция `distance()` предназначена для вычисления разности между двумя итераторами.

```
#include <iterator>
Dist distance (InputIterator pos1, InputIterator pos2)
```

- Возвращает расстояние между итераторами ввода *pos1* и *pos2*.
- Оба итератора должны ссылаться на элементы, принадлежащие одному и тому же контейнеру.
- Если итераторы не являются итераторами произвольного доступа, то позиция итератора *pos2* должна быть достижимой из позиции итератора *pos1*; иначе говоря, итератор *pos2* должен иметь ту же позицию или позицию, расположенную правее.
- Тип возвращаемого значения *Dist* — это тип разности, соответствующий типу итератора.

```
iterator_traits<InputIterator>::difference_type
```

- Подробности изложены в разделе 9.5.

Используя дескрипторы итератора, эта функция выбирает наилучшую реализацию в соответствии с категорией итератора. Для итераторов произвольного доступа эта функция просто возвращает значение выражения *pos2-pos1*. Таким образом, для таких итераторов функция `distance()` имеет константную сложность. Для всех других категорий итераторов выполняется многократный инкремент итератора *pos1*, пока он не достигнет позиции итератора *pos2*, и после этого возвращается количество выполненных инкрементов. Следовательно, для всех остальных категорий итераторов функция `distance()` имеет линейную сложность. Итак, функция `distance()` имеет низкую производительность для всех итераторов, кроме итераторов произвольного доступа. Подумайте, как избежать ее применения в своей программе.

Реализация функции `distance()` описана в разделе 9.5.1. Ее использование демонстрирует следующий пример:

```
// iter/distance1.cpp

#include <iterator>
#include <iostream>
#include <list>
#include <algorithm>
using namespace std;

int main()
{
    list<int> coll;

    // вставляем элементы от -3 до 9
    for (int i=-3; i<=9; ++i) {
        coll.push_back(i);
    }
```

```

// ищем элемент, имеющий значение 5
list<int>::iterator pos;
pos = find (coll.begin(), coll.end(), // диапазон
           5);                       // значение
if (pos != coll.end()) {
// вычисляем и выводим разность между текущей позицией и началом
cout << "difference between beginning and 5: "
     << distance(coll.begin(),pos) << endl;
}
else {
    cout << "5 not found" << endl;
}
}

```

После того как итератору `pos` будет присвоена возвращенная функцией `find()` позиция элемента, имеющего значение 5, функция `distance()` использует эту позицию для вычисления разности между нею и началом контейнера. Результат работы программы выглядит так:

```
difference between beginning and 5: 8
```

Для того чтобы иметь возможность изменять типы итераторов и контейнеров, следует использовать функцию `distance()`, а не `-`. Однако если вы используете функцию `distance()`, то не обнаружите снижения производительности, пока не перейдете от итераторов произвольного доступа к другим видам итераторов.

При вычислении разности между итераторами, которые не являются итераторами произвольного доступа, будьте внимательны. Первый итератор должен ссылаться на элемент, расположенный до, а не после элемента, на который ссылается второй итератор. В противном случае последствия будут непредсказуемыми. Если вам неизвестно, какой из итераторов предшествует другому, следует вычислить расстояние между ними и началом контейнера, а затем вычислить разность между этими расстояниями. Однако для этого необходимо знать, к какому контейнеру относится итератор. В противном случае возможны непредсказуемые последствия. Дополнительные аспекты этой проблемы, связанные с подынтервалами, изложены в разделе 6.4.1.

9.3.4. Функция `iter_swap()`

Это простая вспомогательная функция, предназначенная для обмена значений, на которые ссылаются два итератора.

```

#include <algorithm>
void iter_swap (ForwardIterator1 pos1, ForwardIterator2 pos2)

```

- Обменивает значения, на которые ссылаются итераторы `pos1` и `pos2`.
- Итераторы не обязаны иметь одинаковый тип. Однако значения должны допускать присваивание.

Рассмотрим простой пример (функция `PRINT_ELEMENTS()` введена в разделе 6.6):

```

// iter/iterswap1.cpp

#include <iostream>

```

```

#include <list>
#include <algorithm>
#include <iterator>
#include "print.hpp"
using namespace std;

int main()
{
    list<int> coll;

    // вставляем элементы от 1 до 9
    for (int i=1; i<=9; ++i) {
        coll.push_back(i);
    }

    PRINT_ELEMENTS(coll);

    // Обмениваем первое и второе значения
    iter_swap (coll.begin(), next(coll.begin()));
    PRINT_ELEMENTS(coll);

    // Обмениваем первое и последнее значения
    iter_swap (coll.begin(), prev(coll.end()));
    PRINT_ELEMENTS(coll);
}

```

Результат работы программы выглядит следующим образом:

```

1 2 3 4 5 6 7 8 9
2 1 3 4 5 6 7 8 9
9 1 3 4 5 6 7 8 2

```

Отметим, что функции `next()` и `prev()` предусмотрены стандартом C++11, а использование вместо них операторов `++` и `--` допускается не для каждого контейнера.

```

vector<int> coll;
...
iter_swap (coll.begin(), ++coll.begin()); // ОШИБКА: может не компилироваться
...
iter_swap (coll.begin(), --coll.end());    // ОШИБКА: может не компилироваться

```

Эта проблема подробно описана в разделе 9.2.6.

9.4. Адаптеры итераторов

В разделе описываются адаптеры итераторов, содержащиеся в стандартной библиотеке C++. Эти специальные итераторы позволяют алгоритмам выполнять обратный обход в режиме вставки и работать с потоками.

9.4.1. Обратные итераторы

Обратные итераторы (reverse iterators) переопределяют операции инкремента и декремента так, что их семантика становится обратной. Следовательно, если использовать эти итераторы вместо обычных, алгоритмы будут выполняться в обратном направлении.

Большинство контейнерных классов — все, за исключением последовательных списков и неупорядоченных контейнеров, — а также строки обеспечивают возможность использования обратных итераторов для обхода своих элементов. Рассмотрим следующий пример:

```
// iter/reviter1.cpp

#include <iostream>
#include <list>
#include <algorithm>
using namespace std;

void print (int elem)
{
    cout << elem << ' ';
}

int main()
{
    // создаем список элементов от 1 до 9
    list<int> coll = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };

    // выводим все элементы на экран в обычном порядке
    for_each (coll.begin(), coll.end(), // диапазон
              print);                  // операция
    cout << endl;

    // выводим все элементы на экран в обратном порядке
    for_each (coll.rbegin(), coll.rend(), // диапазон
              print);                    // операция
    cout << endl;
}
```

Функции-члены `rbegin()` и `rend()` возвращают обратный итератор. Аналогично функциям `begin()` и `end()`, эти итераторы определены на полуоткрытом диапазоне. Однако они действуют в обратном направлении.

- Функция-член **`rbegin()`** возвращает позицию первого элемента при обратном обходе. Следовательно, она возвращает позицию последнего элемента.
- Функция-член **`rend()`** возвращает позицию, расположенную за последним элементом при обратном обходе. Следовательно, она возвращает позицию, *предшествующую* первому элементу.

Результат работы программы выглядит следующим образом:

```
1 2 3 4 5 6 7 8 9
9 8 7 6 5 4 3 2 1
```

Стандарт C++11 предусматривает соответствующие функции-члены `crbegin()` и `crend()`, возвращающие обратные итераторы только для чтения. Поскольку они позволяют только читать элементы, их можно (и нужно) использовать в этом примере:

```
// выводим все элементы в обратном порядке
for_each (coll.crbegin(), coll.crend(), // диапазон
          print);                      // операция
```

Итераторы и обратные итераторы

Обычные итераторы можно превратить в обратные. Естественно, эти итераторы должны быть двунаправленными, но обратите внимание, что логическая позиция итератора при его преобразовании перемещается. Рассмотрим следующую программу:

```
// iter/reviter2.cpp

#include <iterator>
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main()
{
    // создаем список с элементами от 1 до 9
    vector<int> coll = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };

    // ищем позицию элемента, имеющего значение 5
    vector<int>::const_iterator pos;
    pos = find (coll.cbegin(), coll.cend(),
               5);

    // выводим на экран значение, на которое ссылается итератор pos
    cout << "pos: " << *pos << endl;

    // преобразовываем итератор pos в обратный итератор rpos
    vector<int>::const_reverse_iterator rpos(pos);

    // выводим на экран значение, на которое ссылается итератор rpos
    cout << "rpos: " << *rpos << endl;
}
```

Эта программа выводит на экран следующий результат:

```
pos: 5
rpos: 4
```

Таким образом, при выводе значения итератора и его преобразовании в обратный итератор это значение изменилось. Это не ошибка, а особенность! Это поведение является следствием того факта, что диапазоны являются полуоткрытыми. Для того чтобы определить все элементы в контейнере, необходимо использовать позицию, расположенную за последним элементом. Однако для обратного итератора этой позицией является позиция,

предшествующая первому элементу. К сожалению, эта позиция может не существовать. Контейнеры не обязаны гарантировать, что позиция, предшествующая первому элементу, является корректной. Представьте себе, что обычные строки и массивы тоже контейнеры, а язык программирования не гарантирует, что адресация массивов не начинается с нулевого адреса...

В результате разработчики обратных итераторов использовали трюк: они “физически” изменили на противоположный “принцип полуоткрытости”. Физически в диапазоне определены обратные итераторы, начало *не включено* в диапазон, а конец *включен*. Однако логически итераторы работают как обычно. Следовательно, существует различие между физической позицией, определяющей элемент, на который ссылается итератор, и логической позицией, определяющей значение, на которое ссылается итератор (рис. 9.3). Вопрос заключается в том, что происходит во время превращения итератора в обратный итератор? Какую позицию сохраняет итератор: логическую (значение) или физическую (элемент)? Как показывает предыдущий пример, имеет место второй вариант. Таким образом, значение перемещается на предыдущий элемент (рис. 9.4).

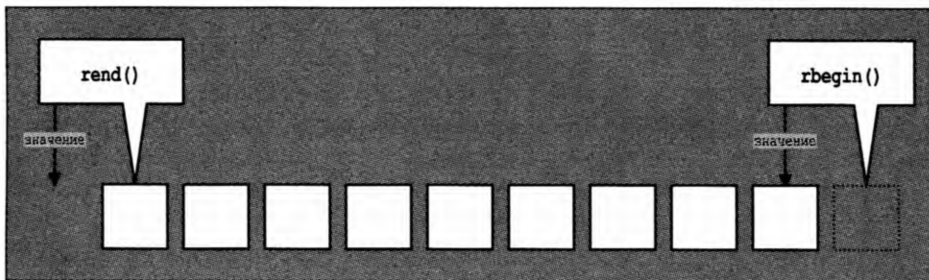


Рис. 9.3. Позиция и значение обратных итераторов

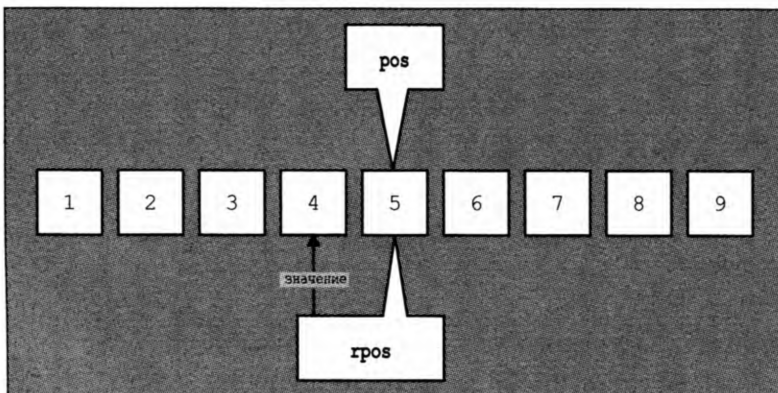


Рис. 9.4. Преобразование итератора *pos* в обратный итератор *rpos*

Вы поняли это решение? У него есть одно преимущество: не надо ничего делать, чтобы преобразовать диапазон, который определен двумя, а не одним итератором. Все элементы остаются корректными. Рассмотрим следующий пример:

```
// iter/reviter3.cpp

#include <iterator>
#include <iostream>
#include <deque>
#include <algorithm>
using namespace std;

void print (int elem)
{
    cout << elem << ' ';
}

int main()
{
    // создаем дек с элементами от 1 до 9
    deque<int> coll = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };

    // ищем позицию элемента со значением 2
    deque<int>::const_iterator pos1;
    pos1 = find (coll.cbegin(), coll.cend(), // диапазон
                2);                          // значение

    // ищем элемент со значением 7
    deque<int>::const_iterator pos2;
    pos2 = find (coll.cbegin(), coll.cend(), // диапазон
                7);                          // значение

    // выводим на экран все элементы из диапазона [pos1,pos2)
    for_each (pos1, pos2, // диапазон
              print);     // операция
    cout << endl;

    // превращаем итераторы в обратные
    deque<int>::const_reverse_iterator rpos1(pos1);
    deque<int>::const_reverse_iterator rpos2(pos2);

    // выводим на экран все элементы из диапазона [pos1,pos2) в обратном порядке
    for_each (rpos2, rpos1, // диапазон
              print);       // операция
    cout << endl;
}
```

Итераторы `pos1` и `pos2` определяют полуоткрытый диапазон, включая элемент со значением 2, но исключая элемент со значением 7. Когда итератор, описывающий этот диапазон, превращается в обратный, диапазон остается корректным и может обрабатываться в обратном порядке. Таким образом, программа выводит на экран следующие результаты:

```
2 3 4 5 6
6 5 4 3 2
```

Итак, функция-член `rbegin()` — это просто

```
контейнер::reverse_iterator(end())
а функция-член rend() — это просто
контейнер::reverse_iterator(begin())
```

Разумеется, константные итераторы могут преобразовываться в тип `const_reverse_iterator`.

Обратное преобразование обратных итераторов с помощью функции-члена `base()`

Обратные итераторы можно превратить обратно в обычные. Для этого обратные итераторы снабжены функцией-членом `base()`.

```
namespace std {
    template <typename Iterator>
    class reverse_iterator ... {
        ...
        Iterator base() const;
        ...
    };
}
```

Рассмотрим пример использования функции `base()`:

```
// iter/reviter4.cpp

#include <iterator>
#include <iostream>
#include <list>
#include <algorithm>
using namespace std;

int main()
{
    // создаем список с элементами от 1 до 9
    list<int> coll = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };

    // ищем позицию элемента, имеющего значение 5
    list<int>::const_iterator pos;
    pos = find (coll.cbegin(), coll.cend(), // диапазон
               5);                        // значение

    // выводим на экран этот элемент
    cout << "pos: " << *pos << endl;

    // превращаем итератор в обратный итератор
    list<int>::const_reverse_iterator rpos(pos);

    // выводим на экран значение элемента, на который ссылается обратный итератор
    cout << "rpos: " << *rpos << endl;

    // превращаем обратный итератор в обычный
    list<int>::const_iterator rrpos;
    rrpos = rpos.base();

    // выводим на экран значение элемента, на который ссылается обычный итератор
```



```
cout << "rrpos: " << *rrpos << endl;
}
```

Эта программа выводит на экран следующие строки:

```
pos: 5
rpos: 4
rrpos: 5
```

Таким образом, преобразование итератора с помощью функции `base()`

```
*rpos.base()
```

эквивалентно его превращению в обратный итератор. Иначе говоря, физическая позиция (элемент итератора) сохраняется, а логическая (значение элемента) перемещается. Другой пример использования функции `base()` приведен в разделе 11.5.3.

9.4.2. Итераторы вставки

Итераторы вставки — это адаптеры итераторов, преобразовывающие присвоение нового значения во вставку этого нового значения. Используя итераторы вставки, алгоритмы могут вставлять элементы, а не заменять их. Все итераторы вставки относятся к категории итераторов вывода. Таким образом, они обеспечивают только возможность присваивать новые значения (см. раздел 9.2.1).

Функциональные возможности итераторов вставки

Как правило, алгоритмы присваивают значения итераторам получателя. Например, рассмотрим алгоритм `copy()` (он описан в разделе 11.6.1):

```
namespace std {
    template <typename InputIterator, typename OutputIterator>
    OutputIterator copy (InputIterator from_pos, // начало источника
                        InputIterator from_end, // конец источника
                        OutputIterator to_pos) // начало получателя
    {
        while (from_pos != from_end) {
            *to_pos = *from_pos; // копируем значения
            ++from_pos;          // инкрементируем итераторы
            ++to_pos;
        }
        return to_pos;
    }
}
```

Этот цикл выполняется, пока текущая позиция итератора источника не достигнет его конца. В цикле итератор источника `from_pos` присваивается итератору получателя `to_pos`, и оба итератора инкрементируются.

Представляет интерес часть, выполняющая присвоение нового значения:

```
*to_pos = value
```

Итератор вставки преобразовывает это присваивание во вставку. Однако эта операция состоит из двух: во-первых, оператор `*` возвращает текущий элемент итератора, во-вторых, оператор `=` присваивает новое значение. Реализации итераторов вставки обычно используют двухэтапный трюк.

- 1. Оператор `*` реализуется как фиктивная операция, просто возвращающая `*this`. Таким образом, для итераторов вставки значение `*pos` эквивалентно `pos`.
- 2. Оператор присваивания реализуется так, что он превращается во вставку. Фактически итератор вставки вызывает функции-члены контейнера `push_back()`, `push_front()` или `insert()`.

Таким образом, чтобы вставить новый элемент с помощью итератора вставки, можно писать `pos = value`, а не `*pos = value`. Однако это касается деталей реализации итераторов ввода. Правильным выражением для присваивания нового значения является `*pos = value`.

Аналогично операция инкремента реализуется как фиктивная операция, просто возвращающая `*this`. Следовательно, модифицировать позицию итератора вставки невозможно. Все операции над итераторами вставки приведены в табл. 9.7.

Таблица 9.7. Операции над итераторами вставки

Выражение	Результат
<code>*iter</code>	Фиктивный (возвращает <code>iter</code>)
<code>iter = value</code>	Вставляет значение
<code>++iter</code>	Фиктивный (возвращает <code>iter</code>)
<code>iter++</code>	Фиктивный (возвращает <code>iter</code>)

Виды итераторов вставки

Стандартная библиотека C++ содержит три вида итераторов вставки: итераторы вставки в конец, итераторы вставки в начало и общие итераторы вставки. Они различаются позициями, в которые выполняется вставка. Фактически каждый из них использует разные функции-члены контейнера. Следовательно, итератор вставки всегда должен быть инициализирован своим контейнером.

Каждый вид итератора вставки имеет удобную функцию для его создания и инициализации. В табл. 9.8 приведены разновидности итераторов вставки и их возможности.

Таблица 9.8. Виды итераторов вставки

Имя	Класс	Вызываемая функция	Создание
Итератор вставки в конец	<code>back_insert_iterator</code>	<code>push_back(value)</code>	<code>back_inserter(cont)</code>
Итератор вставки в начало	<code>front_insert_iterator</code>	<code>push_front(value)</code>	<code>front_inserter(cont)</code>
Общий итератор вставки	<code>insert_iterator</code>	<code>insert(pos, value)</code>	<code>inserter(cont, pos)</code>

Разумеется, контейнер должен иметь функцию-член, которую вызывает итератор вставки, в противном случае этот вид итераторов вставки использовать нельзя. По этой причине итераторы вставки в конец доступны только для векторов, деков, списков и строк, а итераторы вставки в начало — только для деков и списков.

Итераторы вставки в конец

Итератор вставки в конец (back inserter) добавляет значение в конец контейнера, вызывая функцию-член `push_back()` (описание этой функции см. в разделе 8.7.1). Функция `push_back()` доступна только для векторов, деков, списков и строк, поэтому это единственные контейнеры из стандартной библиотеки C++, для которых доступны итераторы вставки в конец.

Итератор вставки в конец должен быть инициализирован своим контейнером в момент своего создания. Функция `back_inserter()` обеспечивает удобный способ для этого. Использование итераторов вставки в конец демонстрирует следующий пример:

```
// iter/backinserter1.cpp
```

```
#include <vector>
#include <algorithm>
#include <iterator>
#include "print.hpp"
using namespace std;

int main()
{
    vector<int> coll;

    // создаем итератор вставки в конец для вектора coll
    // - неудобный способ
    back_insert_iterator<vector<int> > iter(coll);

    // вставка элементов с помощью интерфейса обычных итераторов
    *iter = 1;
    iter++;
    *iter = 2;
    iter++;
    *iter = 3;
    PRINT_ELEMENTS(coll);

    // создаем итератор вставки в конец и вставляем элемент
    // - удобный способ
    back_inserter(coll) = 44;
    back_inserter(coll) = 55;
    PRINT_ELEMENTS(coll);

    // снова используем итератор вставки в конец для добавления новых элементов
    // - резервируем достаточный объем памяти,
    // чтобы избежать ее повторного выделения
    coll.reserve(2*coll.size());
    copy(coll.begin(), coll.end(), // источник
          back_inserter(coll));    // получатель
```

```
    PRINT_ELEMENTS(coll);  
}
```

Результат работы этой программы выглядит следующим образом:

```
1 2 3  
1 2 3 44 55  
1 2 3 44 55 1 2 3 44 55
```

Отметим, что перед вызовом функции `copy()` следует зарезервировать достаточный объем памяти. Причина заключается в том, что итератор вставки в конец вставляет элементы, что может сделать некорректными все остальные итераторы в том же векторе. Следовательно, если зарезервирован недостаточный объем памяти, алгоритм сделает некорректными переданные итераторы источника.

Строки также имеют контейнерный интерфейс STL, включая функцию-член `push_back()`. Следовательно, ее можно использовать для добавления символов в строку. Пример приведен в разделе 13.2.14.

Итераторы вставки в начало

Итератор вставки в начало (`front inserter`) вставляет значение в начало контейнера, вызывая функцию-член контейнера `push_front()` (подробное описание этой функции см. в разделе 8.7.1). Функция-член `push_front()` доступна только для деков, списков и последовательных списков, поэтому это единственные контейнеры из стандартной библиотеки C++, для которых доступны итераторы вставки в начало.

Итератор вставки в начало должен инициализироваться контейнером в момент своего создания. Функция `front_inserter()` обеспечивает удобный способ для этого. Использование итераторов вставки в начало демонстрирует следующий пример:

```
// iter/frontinserter1.cpp  
  
#include <list>  
#include <algorithm>  
#include <iterator>  
#include "print.hpp"  
using namespace std;  
  
int main()  
{  
    list<int> coll;  
    // создаем итератор вставки в начало для списка coll  
    // - неудобный способ  
    front_insert_iterator<list<int> > iter(coll);  
  
    // вставляем элементы с помощью интерфейса обычного итератора  
    *iter = 1;  
    iter++;  
    *iter = 2;  
    iter++;  
    *iter = 3;  
    PRINT_ELEMENTS(coll);
```

```
// создаем итератор вставки в начало и вставляем элементы
// - удобный способ
front_inserter(coll) = 44;
front_inserter(coll) = 55;
PRINT_ELEMENTS(coll);

// используем итератор для вставки всех элементов
copy (coll.begin(), coll.end(), // источник
      front_inserter(coll));    // получатель
PRINT_ELEMENTS(coll);
}
```

Результат работы программы выглядит следующим образом:

```
3 2 1
55 44 3 2 1
1 2 3 44 55 55 44 3 2 1
```

Отметим, что итератор вставки в начало вставляет несколько элементов в обратном порядке. Это объясняется тем, что следующий элемент всегда вставляется перед предыдущим.

Общие итераторы вставки

Общий итератор вставки (*general inserter*)¹ инициализируется двумя значениями: контейнером и позицией для вставки. Используя оба аргумента, итератор вставки вызывает функцию-член `insert()`, передавая ей заданную позицию как аргумент. Функция-член `inserter()` обеспечивает удобный способ для создания и инициализации общего итератора вставки.

Общий итератор вставки доступен для всех стандартных контейнеров (за исключением массивов и последовательных списков), поскольку все они имеют требуемую функцию-член `insert()` (см. раздел 8.7.1). Однако для ассоциативных и неупорядоченных контейнеров позиция используется только как подсказка, потому что в них конкретная позиция определяется значением элемента (см. раздел 8.7.1).

После вставки общий итератор вставки получает позицию нового вставленного элемента. В частности, выполняется такой код:

```
pos = container.insert(pos,value);
++pos;
```

Присваивание значения, возвращенного функцией `insert()`, гарантирует, что позиция итератора всегда является корректной. Без присваивания новой позиции в деках, векторах и строках общий итератор вставки делает сам себя некорректным. Это объясняется тем, что при каждой вставке все итераторы, ссылающиеся на контейнер, становятся некорректными (по крайней мере, это возможно).

Использование общих итераторов вставки иллюстрируется следующим примером:

```
// iter/inserter1.cpp

#include <set>
```

¹ Общий итератор вставки часто просто называют *итератор вставки*.

```
#include <list>
#include <algorithm>
#include <iterator>
#include "print.hpp"
using namespace std;

int main()
{
    set<int> coll;
    // создаем итератор вставки для множества coll
    // - неудобный способ
    insert_iterator<set<int> > iter(coll, coll.begin());

    // вставляем элементы с помощью интерфейса обычного итератора
    *iter = 1;
    iter++;
    *iter = 2;
    iter++;
    *iter = 3;
    PRINT_ELEMENTS(coll, "set: ");

    // создаем итератор вставки и вставляем элемент
    // - удобный способ
    inserter(coll, coll.end()) = 44;
    inserter(coll, coll.end()) = 55;
    PRINT_ELEMENTS(coll, "set: ");

    // используем итератор для вставки всех элементов в список
    list<int> coll2;
    copy (coll.begin(), coll.end(),          // источник
          inserter(coll2, coll2.begin())); // получатель
    PRINT_ELEMENTS(coll2, "list: ");

    // используем итератор вставки для повторной вставки всех элементов
    // в список перед вторым элементом
    copy (coll.begin(), coll.end(),          // источник
          inserter(coll2, ++coll2.begin())); // получатель
    PRINT_ELEMENTS(coll2, "list: ");
}
```

Результат работы программы выглядит следующим образом:

```
set: 1 2 3
set: 1 2 3 44 55
list: 1 2 3 44 55
list: 1 1 2 3 44 55 2 3 44 55
```

Вызовы функции `copy()` показывают, что общий итератор вставки сохраняет порядок следования элементов. Второй вызов функции `copy()` использует конкретную позицию в диапазоне, который передается как аргумент.

Пользовательские итераторы вставки для ассоциативных контейнеров

Как указывалось ранее, при работе с ассоциативными контейнерами аргумент общих итераторов вставки, задающий позицию, рассматривается только как подсказка. Эта подсказка может повысить быстродействие, но может и снизить его. Например, если вставленные элементы следуют в обратном порядке, то эта подсказка может замедлить работу программы, так как поиск правильной позиции для вставки всегда будет начинаться с неправильной точки. Таким образом, неверная подсказка может оказаться хуже, чем отсутствие подсказки. Это хороший пример, демонстрирующий необходимость расширения стандартной библиотеки C++. Пример такого расширения описан в разделе 9.6.

9.4.3. Итераторы потоков

Итератор потока (stream iterator) — это адаптер итератора, позволяющий использовать поток в качестве источника или получателя данных для алгоритмов. В частности, итератор потока ввода можно использовать для чтения элементов из потока ввода, а итератор потока вывода — для записи элементов в поток вывода.

Особой формой итератора потока является итератор буфера потока (stream buffer iterator), который можно использовать для чтения или записи в буфер потока. Итераторы буферов потока обсуждаются в разделе 15.13.2.

Итераторы потока вывода

Итераторы потока вывода (ostream iterator) записывают присвоенные значения в поток вывода. С помощью итераторов потока вывода алгоритмы могут записывать данные непосредственно в потоки. Реализация итератора потока вывода использует ту же концепцию, что и реализация итераторов вставки (см. раздел 9.4.2). Единственное отличие заключается в том, что они преобразовывают присваивание нового значения в операцию вывода с помощью оператора `<<`. Таким образом, алгоритмы могут непосредственно записывать данные в потоки с помощью обычного интерфейса итераторов. Операции над итераторами потоков вывода приведены в табл. 9.9.

Таблица 9.9. Операции над итераторами ostream

Выражение	Результат
<code>ostream_iterator<T>(ostream)</code>	Создает итератор потока вывода для потока <i>ostream</i>
<code>ostream_iterator<T>(ostream, delim)</code>	Создает итератор потока вывода для потока <i>ostream</i> с разделителем, заданным строкой <i>delim</i> (строка <i>delim</i> имеет тип <code>const char*</code>)
<code>*iter</code>	Фиктивная операция (возвращает итератор <i>iter</i>)
<code>iter = value</code>	Записывает значение в поток <i>ostream</i> : <i>ostream</i> <code><<value</code> (после значения выводится разделитель <i>delim</i> , если таковой задан)
<code>++iter</code>	Фиктивная операция (возвращает итератор <i>iter</i>)
<code>iter++</code>	Фиктивная операция (возвращает итератор <i>iter</i>)

При создании итератора потока вывода необходимо указать поток вывода, в который будут записываться данные. С помощью необязательного аргумента можно указать строку, которая будет использоваться как разделитель между отдельными значениями. Без разделителя элементы будут просто непосредственно следовать друг за другом.

Итераторы потока вывода определяются для некоторого типа элементов T.

```
namespace std {
    template <typename T,
              typename charT = char,
              typename traits = char_traits<charT> >
    class ostream_iterator;
}
```

Необязательный второй и третий шаблонные аргументы задают тип потока (см. раздел 15.2.1)².

Использование итераторов потоков вывода иллюстрируется следующим примером:

```
// iter/ostreamiter1.cpp

#include <iostream>
#include <vector>
#include <algorithm>
#include <iterator>
using namespace std;

int main()
{
    // создаем итератор потока вывода для потока cout
    // - значения разделяются символом перехода на новую строку
    ostream_iterator<int> intWriter(cout, "\n");

    // записываем элементы с помощью обычного интерфейса итераторов
    *intWriter = 42;
    intWriter++;
    *intWriter = 77;
    intWriter++;
    *intWriter = -5;

    // создаем коллекцию элементов от 1 до 9
    vector<int> coll = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };

    // записываем все элементы без разделителей
    copy (coll.cbegin(), coll.cend(),
          ostream_iterator<int>(cout));
    cout << endl;

    // записываем все элементы с разделителем " < "
    copy (coll.cbegin(), coll.cend(),
          ostream_iterator<int>(cout, " < "));
    cout << endl;
}
```

² В более старых системах необязательные шаблонные аргументы для типа потока отсутствуют.

Результат работы этой программы выглядит следующим образом:

```
42
77
-5
123456789
1 < 2 < 3 < 4 < 5 < 6 < 7 < 8 < 9 <
```

Отметим, что разделитель имеет тип `const char*`. Следовательно, если передать объект типа `string`, необходимо вызывать его функцию-член `c_str()` (см. раздел 13.3.6), чтобы преобразовать его в правильный тип. Например:

```
string delim;
...
ostream_iterator<int>(cout, delim.c_str());
```

Итераторы потоков ввода

Итераторы потоков ввода (`istream iterator`) — это противоположность итераторов потоков вывода. Итератор потока ввода читает элементы из потока ввода. Используя итераторы потока ввода, алгоритмы могут непосредственно читать данные из потоков. Однако итераторы потоков ввода немного сложнее итераторов потоков вывода (поскольку чтение, как обычно, немного сложнее, чем запись).

В момент создания итератор потока ввода инициализируется потоком ввода, из которого он будет читать данные. Затем с помощью обычного интерфейса итераторов ввода (см. раздел 9.2.2) итератор потока ввода читает элемент за элементом, используя оператор `>>`. Однако чтение может завершиться неудачей (из-за конца файла или ошибки), а диапазон источника данных для алгоритмов должен иметь “конечную позицию”. Для решения этих проблем можно использовать *итератор конца потока* (`end-of-stream iterator`), созданный с помощью конструктора по умолчанию итератора потока ввода. Если чтение завершается неудачей, итератор потока ввода превращается в итератор конца потока. Следовательно, после каждой попытки чтения необходимо сравнивать итератор потока ввода с итератором конца потока, чтобы убедиться, что итератор имеет корректное значение. Все операции над итераторами потока ввода перечислены в табл. 9.10.

Таблица 9.10. Операции над итераторами `istream`

Выражение	Действие
<code>istream_iterator<T>()</code>	Создает итератор конца потока
<code>istream_iterator<T>(istream)</code>	Создает итератор потока ввода для потока <i>istream</i> (и может считывать первое значение)
<code>*iter</code>	Возвращает считанное ранее значение (считывает первое значение, если оно не было считано конструктором)
<code>iter->member</code>	Возвращает член ранее прочитанного значения, если таковой имеется
<code>++iter</code>	Считывает следующее значение и возвращает его позицию
<code>iter++</code>	Считывает следующее значение и возвращает итератор для предыдущего значения
<code>iter1 == iter2</code>	Проверяет, равны ли итераторы <i>iter1</i> и <i>iter2</i>
<code>iter1 != iter2</code>	Проверяет, не равны ли итераторы <i>iter1</i> и <i>iter2</i>

Отметим, что конструктор итератора потока ввода открывает поток и обычно считывает первый элемент. В противном случае он не мог бы вернуть первый элемент при выполнении операции `*` после инициализации. Однако реализации могут отложить чтение первого значения до первого вызова операции `*`. Таким образом, итератор потока ввода не следует определять до того, как он потребуется.

Итераторы потоков ввода определены для некоторого элемента типа `T`.

```
namespace std {
    template <typename T,
              typename charT = char,
              typename traits = char_traits<charT>,
              typename Distance = ptrdiff_t>
    class istream_iterator;
}
```

Необязательные второй и третий шаблонные аргументы задают тип используемого потока (см. раздел 15.2.1). Необязательный четвертый шаблонный элемент задает тип разности между итераторами³.

Два потоковых итератора ввода считаются равными, если

- они являются итераторами конца потока и, следовательно, больше не могут читать данные, или
- оба итератора могут читать и использовать один и тот же поток.

Следующий пример демонстрирует операции, предусмотренные для потоковых итераторов ввода:

```
// iter/istreamiter1.cpp

#include <iostream>
#include <iterator>
using namespace std;

int main()
{
    // создаем итератор потока ввода, читающий целые числа из потока cin
    istream_iterator<int> intReader(cin);

    // создаем итератор конца потока
    istream_iterator<int> intReaderEOF;
    // читаем лексемы с помощью потокового итератора ввода, пока это возможно
    // - записываем их дважды
    while (intReader != intReaderEOF) {
        cout << "once:      " << *intReader << endl;
        cout << "once again: " << *intReader << endl;
        ++intReader;
    }
}
```

Если выполнить эту программы с входными данными

```
1 2 3 f 4
```

³В более старых системах, не использовавших шаблонные параметры, заданные по умолчанию, необязательный четвертый шаблонный аргумент был вторым, а аргументы для типа потока отсутствовали.

то результат ее работы будет выглядеть так:

```
once: 1
once again: 1
once: 2
once again: 2
once: 3
once again: 3
```

Легко видеть, что ввод символа `f` завершает работу программы. Из-за ошибки формата поток теряет корректное состояние. Следовательно, итератор потока ввода `intReader` равен итератору конца потока `intReaderEOF`. Итак, условие цикла становится равным `false`.

Пример использования потоковых итераторов и функции `advance()`

Следующий пример демонстрирует использование обоих видов потоковых итераторов и вспомогательной функции `advance()`:

```
// iter/advance2.cpp

#include <iterator>
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

int main()
{
    istream_iterator<string> cinPos(cin);
    ostream_iterator<string> coutPos(cout, " ");

    // пока при вводе не будет достигнут конец файла
    // - записываем каждую третью строку
    while (cinPos != istream_iterator<string>()) {

        // игнорируем две следующие строки
        advance (cinPos, 2);

        // читаем и записываем третью строку
        if (cinPos != istream_iterator<string>()) {
            *coutPos++ = *cinPos++;
        }
    }
    cout << endl;
}
```

Функция `advance()` перемещает итератор на другую позицию (см. раздел 9.3.1). Используемая с итераторами потоков ввода, функция `advance()` пропускает вводимые лексемы. Например, если поток ввода содержит следующие строки⁴:

⁴Благодарю Эндрю Кёнига (Andrew Koenig) за остроумный пример входных данных.

No one objects if you are doing
a good programming job for
someone whom you respect.

то на выводе получим:

objects are good for you

Не забудьте проверить, остался ли корректным итератор потока ввода после вызова функции `advance()` и перед обращением к значению с помощью выражения `*cinPos`. Применение операции `*` к итератору конца потока приводит к непредсказуемым последствиям.

Другие примеры, демонстрирующие использование алгоритмами итераторов потоков для чтения и записи данных, приведены в разделах 6.5.2, 11.6.1 и 11.7.2.

9.4.4. Итераторы перемещения

В стандарте C++11 имеется адаптер итератора, который превращает любое обращение к элементу в операцию перемещения. Например:

```
std::list<std::string> s;  
...  
std::vector<string> v1(s.begin(), s.end());           // копируем строки в v1  
  
std::vector<string> v2(make_move_iterator(s.begin()), // перемещаем строки в v2  
                      make_move_iterator(s.end()));
```

Эти итераторы позволяют алгоритмам перемещать, а не копировать элементы из одного диапазона в другой. Однако стоит заметить, что алгоритм `move()` делает то же самое (см. раздел 11.6.2).

В принципе использование *итератора перемещения* (move iterator) в алгоритмах целесообразно только в том случае, когда алгоритмы перемещают элементы из диапазона-источника в диапазон-получатель. Кроме того, необходимо гарантировать, что обращение к каждому элементу выполняется только один раз. В противном случае содержимое было бы перемещено несколько раз, что вызвало бы непредсказуемые последствия.

Отметим, что единственная категория итераторов, гарантирующая, что элементы считываются или обрабатываются только один раз, — это категория итераторов ввода (см. раздел 9.2.2). Таким образом, использование итераторов перемещения обычно целесообразно, только если алгоритм имеет источник, в котором требуется итератор ввода, а получатель использует итератор вывода. Единственное исключение — алгоритм `for_each()`, который можно использовать для обработки перемещенных элементов в заданном диапазоне (например, для их перемещения в новый контейнер).

9.5. Свойства итераторов

Разные категории итераторов (см. раздел 9.2) содержат итераторы с разными функциональными возможностями. Было бы полезно и даже необходимо переопределять поведение для разных категорий итераторов. Эту перегрузку можно выполнить с помощью дескрипторов и свойств итераторов (предусмотренных в заголовочном файле `<iterator>`).

Для каждой категории итераторов стандартная библиотека C++ предусматривает *дескриптор итератора* (iterator tag), который можно использовать как “метку” итератора.

```
namespace std {
    struct output_iterator_tag {
    };
    struct input_iterator_tag {
    };
    struct forward_iterator_tag
        : public input_iterator_tag {
    };
    struct bidirectional_iterator_tag
        : public forward_iterator_tag {
    };
    struct random_access_iterator_tag
        : public bidirectional_iterator_tag {
    };
}
```

Обратите внимание на то, что здесь используется наследование. Следовательно, любой однонаправленный итератор, например, *является разновидностью* итератора ввода. Однако дескриптор для однонаправленных итераторов выводится только из дескриптора итератора вывода. Таким образом, любой однонаправленный итератор *не является* разновидностью итератора вывода. Только *модифицирующие* однонаправленные итераторы одновременно удовлетворяют требованиям итераторов вывода (см. раздел 9.2), но для них нет отдельной категории.

При написании обобщенного кода может оказаться важной не только категория итератора. Например, может понадобиться тип элементов, на который ссылается итератор. Для этого стандартная библиотека C++ предоставляет специальную шаблонную структуру для определения *свойств итераторов* (iterator traits), которая содержит всю важную информацию об итераторе и используется как универсальный интерфейс для всех определений типов, необходимых для итераторов (категория, тип элементов и т.д.).

```
namespace std {
    template <typename T>
    struct iterator_traits {
        typedef typename T::iterator_category iterator_category;
        typedef typename T::value_type        value_type;
        typedef typename T::difference_type    difference_type;
        typedef typename T::pointer            pointer;
        typedef typename T::reference          reference;
    };
}
```

В этом шаблоне тип T обозначает тип итератора. Следовательно, можно написать код, который для любого итератора использует его категорию, тип элементов и т.д. Например, следующее выражение определяет тип значения для типа итератора T:

```
typename std::iterator_traits<T>::value_type
```

Эта структура имеет два преимущества:

- гарантирует, что итератор предоставляет определения всех типов;
- ее можно (частично) специализировать для (множества) специальных итераторов.

Второе преимущество реализовано для обычных указателей, которые также можно использовать как итераторы.

```
namespace std {
    template <typename T>
    struct iterator_traits<T*> {
        typedef T                value_type;
        typedef ptrdiff_t        difference_type;
        typedef random_access_iterator_tag iterator_category;
        typedef T*               pointer;
        typedef T&               reference;
    };
}
```

Таким образом, любой тип “указатель на элемент типа T” определен как итератор произвольного доступа. Кроме того, существует частичная специализация (`const T*`) для константных указателей.

Отметим, что итераторы вывода можно использовать только для записи. Следовательно, в итераторе вывода `value_type`, `difference_type`, `pointer` и `reference` можно определить как `void`.

9.5.1. Запись обобщенных функций для итераторов

Используя свойства итератора, можно писать обобщенные функции, которые выводят определения типов или используют разный код реализации в зависимости от категории итератора.

Использование типов итераторов

Простой пример использования свойств итератора — алгоритм, которому необходима временная переменная для элементов. Такое временное значение объявляется примерно так:

```
typename std::iterator_traits<T>::value_type tmp;
```

где T — тип итератора.

Другой пример — алгоритм циклического сдвига элементов.

```
template <typename ForwardIterator>
void shift_left (ForwardIterator beg, ForwardIterator end)
{
    // временная переменная для первого элемента
    typedef typename
        std::iterator_traits<ForwardIterator>::value_type value_type;

    if (beg != end) {
        // сохраняем значение первого элемента
        value_type tmp(*beg);

        // смещаем следующие значения
        ...
    }
}
```

Использование категории итератора

Для использования разных реализаций для разных категорий итераторов необходимо сделать следующее.

- Разрешить шаблонной функции вызывать другую функцию, передавая категорию итератора как дополнительный аргумент. Например:

```
template <typename Iterator>
inline void foo (Iterator beg, Iterator end)
{
    foo (beg, end,
        std::iterator_traits<Iterator>::iterator_category());
}
```

- Реализовать эту другую функцию для произвольной категории итератора, имеющей специальную реализацию, которая не выводится из другой категории итераторов. Например:

```
// foo() для двунаправленных итераторов
template <typename BiIterator>
void foo (BiIterator beg, BiIterator end,
         std::bidirectional_iterator_tag)
{
    ...
}

// foo() для итераторов произвольного доступа
template <typename RaIterator>
void foo (RaIterator beg, RaIterator end,
         std::random_access_iterator_tag)
{
    ...
}
```

Версия для итераторов произвольного доступа, например, использует операции с произвольным доступом, а версия для двунаправленных итераторов нет. Благодаря иерархии дескрипторов итераторов (см. раздел 9.5) можно создать одну реализацию для итераторов нескольких категорий.

Реализация функции `distance()`

Примером выполнения описанной выше двухэтапной процедуры является реализация вспомогательной функции `distance()`, возвращающей расстояние между позициями двух итераторов и их элементов (см. раздел 9.3.3). Реализация итераторов произвольного доступа использует только операцию `-`.

Для всех других категорий итераторов функция возвращает количество инкрементов, необходимых для достижения конца диапазона.

```
// универсальная функция distance()
```

```
template <typename Iterator>
typename std::iterator_traits<Iterator>::difference_type
distance (Iterator pos1, Iterator pos2)
```

```
{
    return distance (pos1, pos2,
                    std::iterator_traits<Iterator>
                        ::iterator_category());
}

// функция distance() для итераторов произвольного доступа
template <typename RaIterator>
typename std::iterator_traits<RaIterator>::difference_type
distance (RaIterator pos1, RaIterator pos2,
          std::random_access_iterator_tag)
{
    return pos2 - pos1;
}

// функция distance() для итераторов ввода,
// одно- и двунаправленных итераторов
template <typename InIterator>
typename std::iterator_traits<InIterator>::difference_type
distance (InIterator pos1, InIterator pos2,
          std::input_iterator_tag)
{
    typename std::iterator_traits<InIterator>::difference_type d;
    for (d=0; pos1 != pos2; ++pos1, ++d) {
        ;
    }
    return d;
}
```

Здесь тип разности между итераторами используется как тип возвращаемого значения. Отметим, что вторая версия использует дескриптор для итераторов ввода, так что эта реализация может использоваться также одно- и двунаправленными итераторами, поскольку их дескрипторы являются производными от класса `input_iterator_tag`.

9.6. Создание пользовательских итераторов

Напишем собственный итератор. Как указано в предыдущем разделе, для этого нам нужны свойства этого итератора. Это можно сделать двумя способами.

1. Написать пять требуемых определений для обобщенной структуры `iterator_traits` (см. раздел 9.5).
2. Написать (частичную) специализацию структуры `iterator_traits`.

В первом случае стандартная библиотека C++ предоставляет специальный базовый класс `iterator<>`, содержащий определения типов. Нужно лишь передать эти типы.

```
class MyIterator
: public std::iterator <std::bidirectional_iterator_tag,
                      type, std::ptrdiff_t, type*, type&> {
    ...
};
```


Первый шаблонный параметр определяет категорию итератора, второй — тип элементов *type*, третий — тип разности, четвертый определяет тип указателя, а пятый — тип ссылки. Последние три аргумента являются необязательными и имеют значения по умолчанию `ptrdiff_t`, *type** и *type&*. Таким образом, часто достаточно использовать следующее определение:

```
class MyIterator
: public std::iterator <std::bidirectional_iterator_tag, type> {
    ...
};
```

Следующий пример демонстрирует, как написать пользовательский итератор. Это итератор вставки для ассоциативных и неупорядоченных массивов. В отличие от итераторов вставки из стандартной библиотеки C++ (см. раздел 9.4.2), в нем не используется позиция вставки:

Рассмотрим реализацию класса итератора:

```
// iter/assoiter.hpp

#include <iterator>

// шаблонный класс для итератора вставки
// для ассоциативных и неупорядоченных массивов

template <typename Container>
class asso_insert_iterator
: public std::iterator <std::output_iterator_tag,
                      typename Container::value_type>
{
    protected:
        Container& container; // контейнер, в который вставляются элементы
    public:
        // конструктор
        explicit asso_insert_iterator (Container& c) : container(c) {

// оператор присваивания
// - вставляет значение в контейнер
        asso_insert_iterator<Container>&
        operator= (const typename Container::value_type& value) {
            container.insert(value);
            return *this;
        }

// разыменование — это фиктивная операция, возвращающая сам итератор
        asso_insert_iterator<Container>& operator* () {
            return *this;
        }

// операция инкремента — это фиктивная операция,
// возвращающая сам итератор
        asso_insert_iterator<Container>& operator++ () {
            return *this;
        }
};
```

```
    }

    asso_insert_iterator<Container>& operator++ (int) {
        return *this;
    }
};

// вспомогательная функция для создания итератора вставки
template <typename Container>
inline asso_insert_iterator<Container> asso_inserter (Container& c)
{
    return asso_insert_iterator<Container>(c);
}
```

Класс `asso_insert_iterator` является производным от класса `iterator`, в котором определены соответствующие типы. Первый шаблонный аргумент — `output_iterator_tag`, который задает категорию итераторов. Второй аргумент — это тип значений, на которые ссылается итератор, т.е. тип `value_type` для контейнера. Поскольку итераторы вывода можно использовать только для записи, это определение типа не является необходимым, поэтому здесь можно передать `void`. Однако передача типа значений, показанная здесь, применима к любой категории итераторов.

В момент создания итератор хранит свой контейнер в переменной `container`, являющейся членом его класса. Любое присвоенное значение вставляется в контейнер с помощью функции `insert()`. Операторы `*` и `++` являются фиктивными и просто возвращают сам итератор. Таким образом, итератор поддерживает управление. Если используется обычный интерфейс итератора

```
*pos = value
```

то выражение `*pos` возвращает значение `*this`, которому присваивается новое значение. Это присваивание преобразовывается в вызов `insert(value)` для контейнера.

После определения класса итераторов вставки определяется обычная вспомогательная функция `asso_inserter` для создания и инициализации итератора вставки. В следующей программе такой итератор вставки используется для вставки элементов в неупорядоченное множество:

```
// iter/assoiter1.cpp

#include <iostream>
#include <unordered_set>
#include <vector>
#include <algorithm>
#include "print.hpp"
#include "assoiter.hpp"

int main()
{
    std::unordered_set<int> coll;
    // создаем итератор вставки для множества coll
    // - неудобный способ
    asso_insert_iterator<decltype(coll)> iter(coll);
```

```

// вставляем элементы с обычным интерфейсом итераторов
*iter = 1;
iter++;
*iter = 2;
iter++;
*iter = 3;
PRINT_ELEMENTS(coll);

// создаем итератор вставки для множества coll и вставляем элементы
// - удобный способ
asso_inserter(coll) = 44;
asso_inserter(coll) = 55;
PRINT_ELEMENTS(coll);

// используем итераторы вставки и алгоритм
std::vector<int> vals = { 33, 67, -4, 13, 5, 2 };
std::copy (vals.begin(), vals.end(),           // источник
           asso_inserter(coll));               // получатель
PRINT_ELEMENTS(coll);
}

```

Обычное применение класса `asso_inserter` демонстрирует вызов алгоритма `copy()`:

```

std::copy (vals.begin(), vals.end(),           // источник
           asso_inserter(coll));               // получатель

```

где выражение `asso_inserter(coll)` создает итератор вставки, вставляющий любой аргумент в множество `coll`, вызывая функцию

```
coll.insert(val).
```

Другие инструкции подробно демонстрируют работу итератора. Программа выводит следующие результаты:

```

1 2 3
55 44 1 2 3
-4 33 55 44 67 1 13 2 3 5

```

Отметим, что этот итератор можно использовать и с ассоциативными контейнерами. Таким образом, если в обеих директивах `include` и объявлении множества `coll` заменить `unordered_set` на `set`, то программа останется работоспособной (хотя элементы в контейнере будут упорядочены).

